

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	N.MANASA	Reg. No.:	192472031
Section / Batch:	CSE-AI	Date:	11/8/26

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:
Document → Chunking → Embedding → Retrieval → Generation

- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

5. Employee Policy QA System: Develop a RAG application using an employee policy document. Answer questions about leave, working hours, attendance, holidays, and benefits.

INTRODUCTION

The Employee Policy Question-Answering System is an Artificial Intelligence application developed using Retrieval-Augmented Generation (RAG). The main purpose of this system is to help employees quickly obtain information from company policy documents.

Traditional employee policy documents can be lengthy and difficult to search manually. This system allows employees to ask questions in natural language, such as “*How many leave days are available?*”, “*What are the working hours?*”, or “*What are the company holidays?*”.

The system extracts information from an employee policy document, divides the document into smaller sections, converts the sections into numerical embeddings, and stores them in a vector database. When a user asks a question, the system retrieves the most relevant policy information and uses it to provide an appropriate answer.

The system can be used for answering questions related to leave policies, working hours, attendance, holidays, and employee benefits. It demonstrates how AI, Natural Language Processing (NLP), embeddings, vector search, and RAG can be combined to create a useful workplace information assistant.

OBJECTIVES

1. To develop an AI-based system for answering employee policy-related questions.
2. To use Retrieval-Augmented Generation (RAG) for retrieving relevant information from policy documents.
3. To process and convert employee policy documents into searchable text chunks.
4. To generate embeddings for the policy information using an embedding model.
5. To store and search the embeddings efficiently using a vector database such as FAISS.
6. To provide quick and relevant answers about leave, working hours, attendance, holidays, and employee benefits.
7. To reduce the time required for employees and HR staff to manually search through lengthy policy documents.
8. To provide a simple and user-friendly interface for interacting with the AI system.

3. Workflow

RAG Workflow

Employee Policy PDF



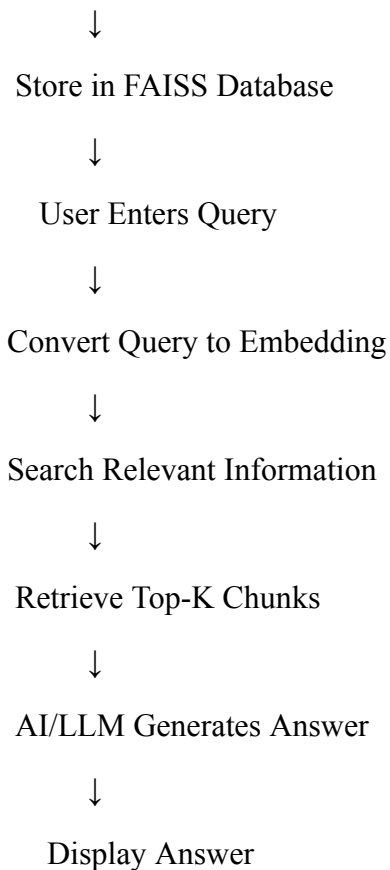
Extract PDF Text



Text Chunking



Generate Embeddings



Step-by-Step Workflow

1. **Document Loading** – The employee policy PDF is loaded into the Python application.
2. **Text Extraction** – Text is extracted from the PDF using the PyPDF library.
3. **Text Chunking** – The extracted text is divided into smaller chunks for efficient processing.
4. **Embedding Generation** – Each text chunk is converted into a numerical vector using Sentence Transformers.
5. **Vector Storage** – The embeddings are stored in a FAISS vector database.
6. **User Query** – The employee enters a question through the application.
7. **Similarity Search** – The question is converted into an embedding and compared with the stored policy embeddings.
8. **Information Retrieval** – The most relevant policy sections are retrieved.
9. **Answer Generation** – The retrieved information is used to generate the final answer.

10. **Result Display** – The answer is displayed to the employee through the application interface.

PROGRAM

```
import streamlit as st
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

# Page configuration
st.set_page_config(
    page_title="Employee Policy QA System",
    page_icon="🤖"
)

st.title("🤖 Employee Policy QA System")
st.write(
    "Ask questions about leave, attendance, "
    "working hours, holidays and benefits."
)

# Read PDF document
def read_pdf(file_path):

    reader = PdfReader(file_path)
```

```
text = ""
```

```
for page in reader.pages:
```

```
    page_text = page.extract_text()
```

```
    if page_text:
```

```
        text += page_text + "\n"
```

```
return text
```

```
# Split document into chunks
```

```
def split_text(text, chunk_size=500):
```

```
    words = text.split()
```

```
    chunks = []
```

```
    for i in range(0, len(words), chunk_size):
```

```
        chunk = " ".join(
```

```
            words[i:i + chunk_size]
```

```
        )
```

```
        chunks.append(chunk)
```

```
return chunks
```

```
# Create vector database

@st.cache_resource
def create_database():

    text = read_pdf("policy.pdf")

    chunks = split_text(text)

    model = SentenceTransformer(
        "all-MiniLM-L6-v2"
    )

    embeddings = model.encode(chunks)

    embeddings = np.array(
        embeddings
    ).astype("float32")

    dimension = embeddings.shape[1]

    index = faiss.IndexFlatL2(
        dimension
    )

    index.add(embeddings)

    return model, index, chunks
```

```
# Load database
model, index, chunks = create_database()
```

```
# User question
question = st.text_input(
    "Enter your question:"
)
```

```
# Retrieve relevant information
if question:
```

```
    query_embedding = model.encode(
        [question]
    )
```

```
    query_embedding = np.array(
        query_embedding
    ).astype("float32")
```

```
    distances, indices = index.search(
        query_embedding,
        3
    )
```

```
    context = ""
```

```
    for i in indices[0]:
```

```
if i < len(chunks):  
    context += chunks[i] + "\n"  
  
# Display retrieved information  
st.subheader("Retrieved Policy Information")  
  
st.write(context)  
  
# Display answer  
st.subheader("Answer")  
  
st.write(  
    "According to the employee policy document:"  
)  
  
st.write(context)
```

RESULT

The Employee Policy Question-Answering System was successfully developed using Python and the Retrieval-Augmented Generation approach. The system was able to read an employee policy document, divide the information into smaller chunks, generate embeddings, and retrieve the most relevant information based on the user's question.

The system can answer questions related to leave, working hours, attendance, holidays, and employee benefits. The use of vector similarity search helps the system identify relevant sections of the policy document quickly.

For example, when the user asks “How many annual leave days are available?”, the system retrieves the relevant leave policy information and displays it as the answer.

The project demonstrates the practical application of Artificial Intelligence, Natural Language Processing, semantic search, embeddings, and RAG in an employee-support system.

CONCLUSION

The Employee Policy Question-Answering System provides an efficient AI-based solution for accessing employee policy information. By using Retrieval-Augmented Generation, the system can retrieve relevant information from policy documents and provide useful answers to employee queries.

The project reduces the need for manual searching of lengthy documents and can improve the efficiency of HR information services. The system can be further enhanced by integrating a powerful Large Language Model, multiple policy documents, multilingual support, voice-based queries, authentication, and cloud deployment.

Overall, the project demonstrates how AI and RAG technology can be applied to build an intelligent employee policy assistant.

26.RAG with Irrelevant Query Detection: Develop a RAG system that identifies questions not covered by the documents and responds appropriately instead of generating unsupported answers.

INTRODUCTION

RAG with Irrelevant Query Detection is an Artificial Intelligence system designed to answer questions using information available in a given set of documents. Retrieval-Augmented Generation (RAG) combines document retrieval with an AI language model to generate answers based on retrieved information.

A major problem with normal RAG systems is that they may generate an answer even when the user's question is not related to the available documents. This can result in unsupported or incorrect information, commonly known as hallucination.

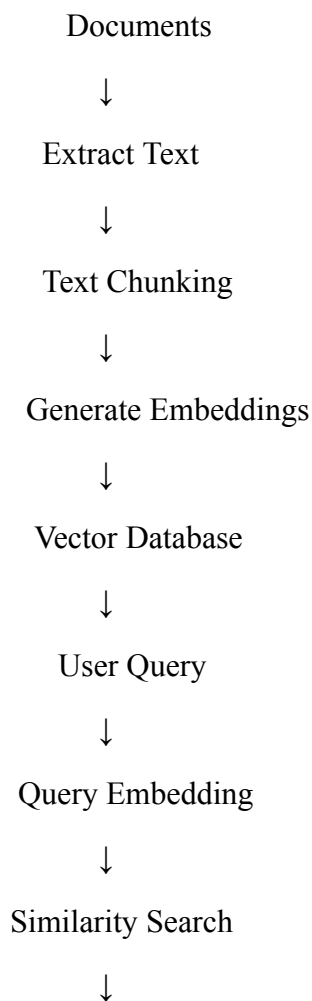
The proposed system addresses this problem by introducing an irrelevant query detection mechanism. When a user asks a question, the system first searches the document database and calculates the similarity between the query and the retrieved document content. If the similarity score is above a predefined threshold, the system generates an answer using the retrieved information. If the similarity score is below the threshold, the system identifies the query as irrelevant and responds that the required information is not available in the documents.

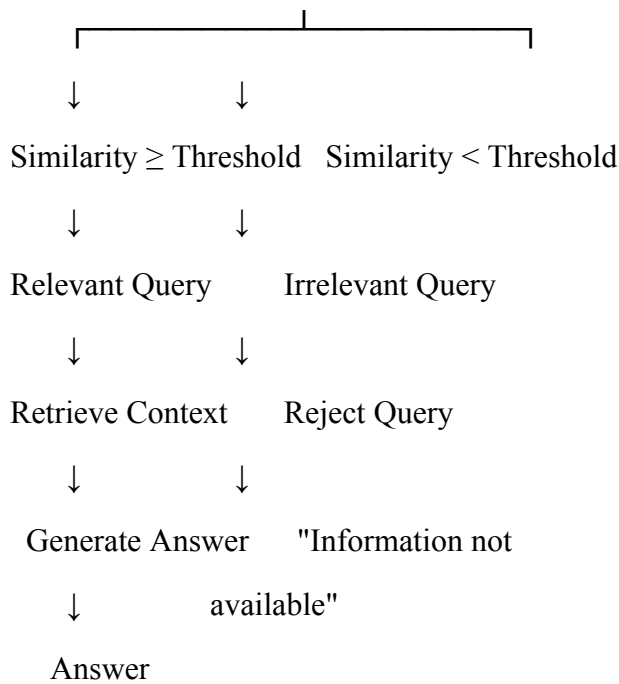
This approach improves the reliability and trustworthiness of RAG-based question-answering systems.

OBJECTIVES

1. To develop a Retrieval-Augmented Generation (RAG) based question-answering system.
2. To retrieve relevant information from a given document collection.
3. To identify queries that are not covered by the available documents.
4. To prevent the AI system from generating unsupported answers.
5. To use semantic similarity for detecting relevant and irrelevant questions.
6. To establish a similarity threshold for query classification.
7. To provide an appropriate response when the required information is unavailable.
8. To improve the accuracy, reliability, and trustworthiness of AI-generated answers.

3. Workflow





Step-by-Step Workflow

Step 1: Document Loading

The system loads the knowledge documents.

Step 2: Text Extraction

Text is extracted from the documents.

Step 3: Text Chunking

The document is divided into smaller chunks.

Step 4: Embedding Generation

Each chunk is converted into a numerical vector using an embedding model.

Step 5: Vector Database

The vectors are stored in FAISS for efficient similarity search.

Step 6: User Query

The user enters a question.

Step 7: Similarity Search

The query is converted into an embedding and compared with document embeddings.

Step 8: Irrelevant Query Detection

The similarity score is compared with a predefined threshold.

Step 9: Answer Generation

- If the query is relevant → retrieve context and generate an answer.
- If the query is irrelevant → do not generate an unsupported answer.

Step 10: Display Result

The system displays either the answer or an appropriate rejection message.

PROGRAM:

```
import streamlit as st

from sentence_transformers import SentenceTransformer

import faiss

import numpy as np


# Page Configuration

st.set_page_config(
    page_title="RAG Irrelevant Query Detection",
    page_icon="🤖"
)

st.title("🤖 RAG with Irrelevant Query Detection")

st.write(
    "This system answers questions only when "
    "relevant information is available in the document."
)


# Load Document

def load_document():
```

```
with open(
    "document.txt",
    "r",
    encoding="utf-8"
) as file:
```

```
    return file.read()
```

```
# Split Text
```

```
def split_text(text):
```

```
    paragraphs = text.split("\n")
```

```
    chunks = []
```

```
    for paragraph in paragraphs:
```

```
        paragraph = paragraph.strip()
```

```
        if paragraph:
```

```
            chunks.append(paragraph)
```

```
    return chunks
```

```
# -----
```

```
# Create Vector Database
```

```
# -----
```

```
@st.cache_resource
```

```
def create_database():
```

```
    text = load_document()
```

```
    chunks = split_text(text)
```

```
    model = SentenceTransformer(
```

```
        "all-MiniLM-L6-v2"
```

```
)
```

```
    embeddings = model.encode(chunks)
```

```
    embeddings = np.array(
```

```
        embeddings
```

```
).astype("float32")
```

```
    # Normalize vectors
```

```
    faiss.normalize_L2(embeddings)
```

```
    dimension = embeddings.shape[1]
```

```
    index = faiss.IndexFlatIP(
```

```
        dimension
```

```
)
```

```
    index.add(embeddings)
```

```
return model, index, chunks
```

```
# -----
```

```
# Load Database
```

```
# -----
```

```
model, index, chunks = create_database()
```

```
# -----
```

```
# User Query
```

```
# -----
```

```
question = st.text_input(
```

```
    "Enter your question:"
```

```
)
```

```
# -----
```

```
# RAG + Irrelevant Query Detection
```

```
# -----
```

```
if question:
```

```
    query_embedding = model.encode(
```

```
        [question]
```

```
    )
```

```
query_embedding = np.array(  
    query_embedding  
).astype("float32")
```

```
faiss.normalize_L2(  
    query_embedding  
)
```

```
# Search top 3 results  
scores, indices = index.search(  
    query_embedding,  
    3  
)
```

```
best_score = scores[0][0]
```

```
best_index = indices[0][0]
```

```
# -----
```

```
# Similarity Threshold
```

```
# -----
```

```
threshold = 0.35
```

```
st.write(  
    "Similarity Score:",
```

```
round(float(best_score), 3)
)
```

```
# -----
# Relevant Query
# -----
```

```
if best_score >= threshold:
```

```
    context = chunks[best_index]
```

```
    st.success(
        "Relevant query detected."
    )
```

```
    st.subheader(
        "Retrieved Information"
    )
```

```
    st.write(context)
```

```
    st.subheader(
        "Answer"
    )
```

```
    st.write(
        "Based on the available document: "
        + context
    )
```

)

Irrelevant Query

else:

st.warning(

"Irrelevant query detected."

)

st.subheader(

"Answer"

)

st.write(

"Sorry, the requested information "

"is not available in the provided documents."

)

RESULT

The RAG system with Irrelevant Query Detection was successfully developed. The system retrieves relevant information from the available documents and calculates a semantic similarity score between the user's query and the stored document content.

A predefined similarity threshold is used to distinguish relevant queries from irrelevant queries. When the similarity score is above the threshold, the system retrieves the relevant information and provides an answer. When the score is below the threshold, the system rejects the query and informs the user that the required information is not available in the provided documents.

The system therefore reduces the possibility of generating unsupported answers and improves the reliability of the RAG application.

CONCLUSION

The RAG with Irrelevant Query Detection system provides a reliable approach for document-based question answering. Unlike a basic RAG system, it does not blindly generate an answer for every question. Instead, it first checks whether the user's question is sufficiently related to the available knowledge.

By using embeddings, vector similarity search, and a predefined threshold, the system can distinguish between relevant and irrelevant queries. This helps reduce hallucinations and prevents unsupported information from being presented as an answer.

The system can be further improved by integrating a Large Language Model, adaptive similarity thresholds, multiple documents, source citations, multilingual support, and a conversational interface. Thus, the project demonstrates an important AI technique for building safer and more trustworthy RAG applications.